

PRODUCT REQUIREMENTS DOCUMENT

CP38 — Business UAT Sign-off Verification

Shared-mailbox, on-demand verification flow — POC scope

Project (example)	PIT-Arnour
Checkpoint	CP38 — Business UAT sign-off obtained
Trigger	On-demand, via copilot query (not webhook-driven)
Status	Draft for engineering handoff

1. Overview

Confirms whether a project's UAT sign-off has been received from the correct, pre-configured authorized owner, via a shared mail inbox, evaluated on demand when a user asks the copilot about it.

This checkpoint uses two statuses only — Compliant or Pending. There is no Non-Compliant outcome at this stage: a mismatched or missing sign-off does not necessarily mean the checkpoint has failed, only that sufficient evidence has not yet been received. Pending outcomes carry a reason field so the dashboard and copilot can distinguish "nothing arrived" from "the wrong person responded" without needing a third status code.

2. Setup — required before this checkpoint can run

- Shared inbox: one common mailbox that receives all UAT sign-off emails (e.g. audit-poc@acmecapital-demo.com). Read-only OAuth access is sufficient.
- Project configuration, stored once per project: required role, authorized signer name, and authorized signer email — never inferred or guessed by the AI.
- Subject-line convention: emails are expected to contain a recognizable pattern (e.g. "PIT-Arnour UAT Sign-off") — this is what the mailbox search matches on, keeping evidence retrieval targeted rather than a full inbox scan.

```
{ project_id: "PIT-Arnour", checkpoint_id: "CP38", required_role: "Business",
  authorized_name: "Nikhil Chawla",
  authorized_email: "nikhil.chawla@company.example" }
```

Dummy test emails to prepare for the demo: one from the authorized address with clear sign-off language (proves the Compliant case), one from a different address (proves the wrong-sender Pending case), and optionally none sent for a given project (proves the no-email Pending case by default).

3. The three cases

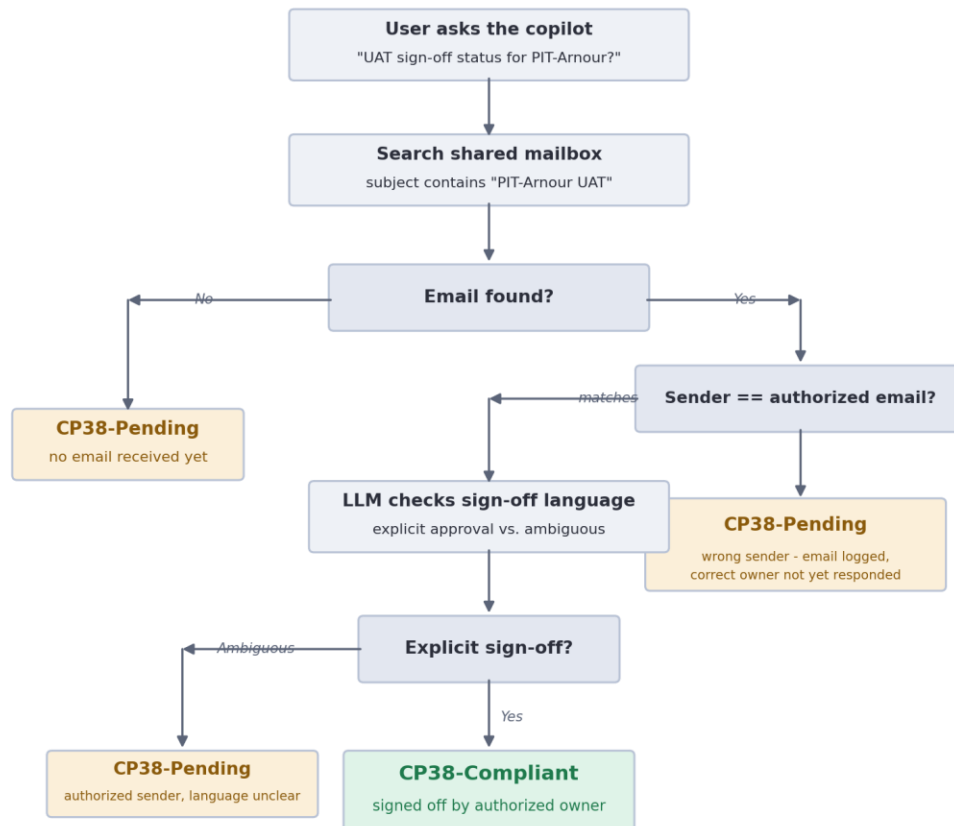
Case	What happened	Status	Reason shown
1	Email from the authorized owner, with explicit sign-off language	CP38-Compliant	Signed off by [name] ([email])

Case	What happened	Status	Reason shown
2	No matching email found at all	CP38-Pending	No sign-off email received yet
3	Email found, but sender is not the authorized owner	CP38-Pending	Received from [wrong email] — not the authorized signer (expected [correct email])

A fourth situation is handled by the same logic without a separate labeled case: if the sender is authorized but the email's language is ambiguous rather than explicit, the result is also CP38-Pending, with the reason "authorized sender, language unclear." This keeps the two-status model intact while still surfacing why the checkpoint has not yet passed.

4. Step-by-step flow

1. User asks the copilot a question, e.g. "What's the status of PIT-Arnour's UAT sign-off?"
2. Agent resolves the request against stored project configuration: project = PIT-Arnour, checkpoint = CP38.
3. Agent searches the shared mailbox for a targeted subject match (e.g. subject contains "PIT-Arnour UAT") — never a full inbox scan.
4. If no matching email is found, the flow stops here: result is CP38-Pending, reason "no email received yet."
5. If an email is found, the deterministic sender check runs first: does the sender exactly match the project's configured authorized email?
6. If the sender does not match, the flow stops here — the LLM step is skipped entirely. Result is CP38-Pending, reason "wrong sender," with the incorrect sender's address recorded.
7. If the sender matches, the LLM checks the email body to classify whether it constitutes explicit sign-off language.
8. If the language is explicit, result is CP38-Compliant. If ambiguous, result is CP38-Pending, reason "authorized sender, language unclear."
9. The result is written to the backend and read directly by the dashboard — no separate frontend logic required.
10. The copilot replies in plain language, matching whichever of the outcomes above was reached.



5. Backend payload

The `pending_reason` field is what allows Case 2 and Case 3 to render differently on the dashboard even though `outcome_id` is identical for both.

```

POST /api/checkpoints/CP38/result
{
  project_id: "PIT-Arnour",
  outcome_id: "CP38-C" | "CP38-Pending",
  pending_reason: null | "no_email" | "wrong_sender" | "ambiguous_language",
  wrong_sender_email: null | "rahul.sharma@company.example",
  confidence: 94, // meaningful only for the Compliant case
  evidence_ref: "email-msg-id-xyz" | null, // null when no email was found at all
  approver_actor: "nikhil.chawla@company.example" | null,
  timestamp: "2026-09-16T10:00:00Z"
}

```

6. Copilot response text, per case

Case 1 — Compliant

CP38 - Business UAT Sign-off: Compliant
Signed off by Nikhil Chawla (nikhil.chawla@company.example), confidence 94%.

Case 2 — Pending, no email

CP38 - Business UAT Sign-off: Pending

No sign-off email has been received yet for PIT-Arnour.

Case 3 — Pending, wrong sender

CP38 - Business UAT Sign-off: Pending

An email was received, but from rahul.sharma@company.example - this is not the authorized signer for PIT-Arnour (expected nikhil.chawla@company.example). Still awaiting sign-off from the correct owner.

7. Why this scope is right for the POC

- The deterministic sender check and the LLM language check stay cleanly separated — sender identity is never an LLM decision, matching the same principle used across every other checkpoint in this build.
- The LLM call is skipped entirely in Case 3 — once the sender fails the identity check, there is no reason to spend a model call classifying language from someone who isn't authorized to sign off in the first place.
- No Jira automation/CC-routing setup is required for this version — emails are manually placed in the shared inbox for the demo, which is a legitimate stand-in at POC stage rather than a shortcut that breaks the architecture.
- Two statuses instead of three keeps both the backend logic and the dashboard simpler, without losing information — the pending_reason field carries the same detail a Non-Compliant status would have, without making a stronger claim than the evidence actually supports.
- Not addressed by design, and worth tracking separately: a checkpoint that stays Pending indefinitely is its own kind of finding once a due date passes. This POC does not implement an overdue/aging state for Pending checkpoints — noted here as a deliberate, known gap rather than an oversight.